

COM3240 Reinforcement Learning Assignment Report

Jude Gibson
22nd May 2024

Abstract

Reinforcement learning is a type of unsupervised machine learning that allows an agent to improve performance based on some reward. Certain actions that an agent takes within its environment can provide either positive or negative rewards, which may incentivise or de-incentivise the same action on subsequent episodes. Algorithms like SARSA or Q-Learning allow the rewards to propagate throughout the agent's environment, meaning that an agent can learn which direction a reward is in without actually receiving the reward. This project compares how the SARSA and Q-Learning algorithms in 3 distinct environments employing both deterministic and stochastic elements.

1. Introduction

Reinforcement learning is an unsupervised machine learning technique where agents must use a performance score or 'reward' to learn its task. Algorithms such as Q-learning or SARSA can be used to propagate rewards throughout an environment so that subsequent episodes give the agent more information about where rewards can be found.

For this project, an agent had to explore a series of 2D grids in search of a reward while avoiding 'lava tiles'. Both the SARSA and Q-learning algorithms were implemented to complete this task and their performances were evaluated.

The main difference between SARSA and Q-Learning is that SARSA is an 'online' algorithm and Q-Learning is 'offline'. This means that SARSA learns from the actions it takes as it does so. Q-Learning learns from the maximum result of the next state, rather than the next action it will take like SARSA. This results in subtle performance differences that I will explore in more detail throughout this paper.

2. Environment 1 - Deterministic Rewards and Transitions

2.1 Overview

The first environment is a relatively simple one. Only a single reward is present and everything is completely deterministic. This means that a simple agent will be able to find the reward fairly quickly. In my experiments agents using both SARSA and Q-Learning were able to converge on the reward in around 50 episodes.

2.2 Parameter Optimisation

When I began to tune the parameters, I started with the same initial values for both SARSA

and Q-Learning. These were: $\gamma = 0.99$, $\alpha = 0.3$ and $\epsilon = 0.1$. I tested both agents and made adjustments to the parameters according to the performance and some heuristics. I used the *plot_directions()* method that I wrote to determine how a parameter set performed. I never changed more than 2 parameters at a time and never γ and α at the same time. This is simply due to the fact that they both have an influence on the Q update method and so adjustments between the two can be difficult to differentiate. I tried to avoid tuning parameters beyond 2 decimal places apart from epsilon, which often ended up at 3 decimal places. Tables 1 and 2 show how I reached the final parameter values. Note that I did more tests than are shown but for brevity, I just show the most important tests.

Due to SARSA's 'online' methodology, it performs better with lower ϵ values. Since Q-Learning learns from the maximum Q-value for the next state, exploration has less of an impact. Therefore it can afford to explore more without affecting the process as much. SARSA does not have this luxury, so ϵ values need to be much more refined.

For similar reasons, SARSA also performs better with higher α values. Since there is less of a focus on exploration, good performance should be reinforced faster than Q-Learning which, as previously established, benefits from more exploration. Inversely, due to the higher exploration rate of Q-Learning, it is more likely to make bad decision, which should be reinforced less.

While SARSA ended up with a lower value for γ , I do not think there is much to be learned from this, because the difference between γ for the 2 algorithms is relatively small. I think the higher value for Q-Learning here can be explained as it compensating for a smaller value for α .

Table 1: A table showing the parameter tuning for SARSA in environment 1

	Test 1	Test 2	Test 3	Test 4	Test 5	Test 6	Test 7	Test 8
Gamma (γ)	0.99	0.6	0.6	0.65	0.67	0.67	0.55	0.58
Alpha (α)	0.3	0.3	0.25	0.25	0.25	0.27	0.27	0.27
Epsilon (ϵ)	0.1	0.05	0.03	0.04	0.04	0.035	0.02	0.025
Best Path Length	17	11	13	13	17	13	11	11
Final Path Length	23	8	13	15	19	13	7	11

Table 2: A table showing the parameter tuning for Q-Learning in environment 1

	Test 1	Test 2	Test 3	Test 4	Test 5	Test 6	Test 7	Test 8
Gamma (γ)	0.99	0.8	0.7	0.6	0.6	0.65	0.65	0.65
Alpha (α)	0.3	0.3	0.25	0.27	0.29	0.29	0.2	0.175
Epsilon (ϵ)	0.1	0.08	0.05	0.05	0.047	0.047	0.055	0.045
Best Path Length	13	13	13	13	15	15	13	11
Final Path Length	13	13	13	15	15	15	13	11

2.3 Results

Figures 1 and 2 show how SARSA performed against Q-Learning over 50 runs. Each episode shown in the graph is averaged over the 50 runs that took place. As you can see, SARSA is generally slightly slower than Q-Learning, however as shown in the second graph it does reach the same performance as Q-Learning given time.

Figure 1: The average number of steps of SARSA and Q-Learning in environment 1

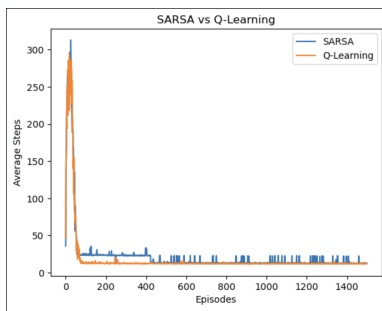


Figure 2: The average rewards of SARSA and Q-Learning in environment 1

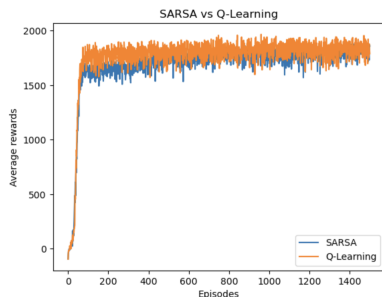


Figure 3: The preferred directions of SARSA and Q-Learning after training

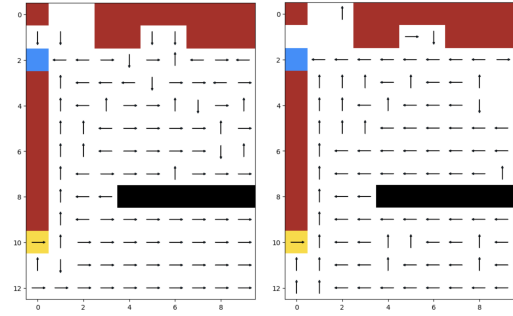


Figure 3 shows the preferred directions of SARSA and Q-Learning respectively. As you can see, they both learn to find the shortest path to the reward without moving into the lava tiles. It is interesting to note that aside from the main route, SARSA seems to focus more on avoiding the lava tiles where Q-Learning focuses on reaching the reward. This is due to the fact that Q-Learning learns from the best action, which would be moving towards the reward.

2.4 Alternate Policy

Figure 4: *The average number of steps of SARSA and Q-Learning using Softmax*

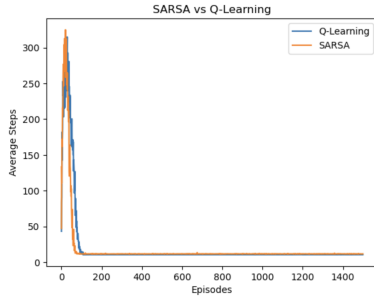
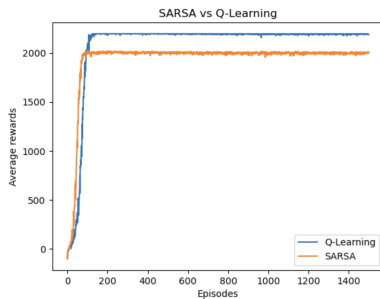


Figure 5: *The average rewards of SARSA and Q-Learning using Softmax*



In the previous examples, I employed the ϵ -greedy policy where the agent selects the optimal action according to some probability. Otherwise the agent picks a uniformly random action. This policy is effective for achieving a balance between exploration and exploitation since ϵ can be precisely tuned according to the task being completed. However the uniform random distribution used means that undesirable actions can be selected as often as better actions, meaning that the exploration itself can be inefficient. One way to get around this is by using a softmax policy. In softmax, the action selection is always random but the distribution is scaled according to an actions desirability. The probability of an action is given by

$$p(a) = \frac{e^{Q(a)/\tau}}{\sum e^{Q(b)/\tau}}$$

where τ is a 'temperature' value. As τ approaches 1, the actions become equally probable and as such, lower values make the agent act closer to a greedy policy.

As you can see, softmax allows the agent to learn much faster and much consistently. The learning in ϵ -greedy is noisy due to the fact that the probability of any action is uniform. Now that the probability is scaled, the learning curve appears more defined. In this example, SARSA is able to learn faster than Q-Learning, however it is not able to exploit rewards as effectively.

Note that given the success of the softmax policy in this environment, I decided to continue using softmax for the remainder of the project.

3. Environment 2 - Stochastic Rewards and Deterministic Transitions

3.1 Overview

The second environment was distinct from the first in that there were two different rewards. One exists in the same location as the first environment, but a second has been added on the opposite end of the environment. This time both locations give random rewards according to normal distributions, the means (μ) of which are 700 and 300 and the standard deviations (σ) are 100 and 70 respectively.

Since the first reward is larger, the agents should therefore learn to travel to the first location. However, while the first reward is larger, there is a chance that the second may appear larger if the agent visits that reward. For this reason, a smaller learning rate may be needed to prevent the agent from learning the path to the second reward too quickly.

3.2 Parameter Optimisation

I followed a similar methodology to the first environment when optimising the parameters for the second environment. The largest difference here was that instead of tuning ϵ I had to tune τ . This is because I decided to continue with the softmax policy, as stated in sec-

Table 3: A table showing the parameter optimisation for SARSA in environment 2

	Test 1	Test 2	Test 3	Test 4	Test 5	Test 6
Gamma (γ)	0.99	0.99	0.7	0.75	0.75	0.77
Alpha (α)	0.3	0.2	0.2	0.15	0.2	0.2
Tau (τ)	2.0	1.5	1.5	1.5	1.25	1.2
Best Path Length	14	13	13	14	11	11
Final Path Length	14	13	13	14	11	11

Table 4: A table showing the parameter optimisation for Q-Learning in environment 2

	Test 1	Test 2	Test 3	Test 4	Test 5	Test 6	Test 7	Test 8
Gamma (γ)	0.99	0.99	0.7	0.75	0.75	0.75	0.7	0.64
Alpha (α)	0.3	0.2	0.2	0.15	0.25	0.18	0.15	0.15
Tau (τ)	2.0	1.5	1.5	1.5	1.2	1.2	1.2	1.25
Best Path Length	15	14	13	14	14	14	13	11
Final Path Length	15	14	13	14	14	14	13	11

tion 2.4. SARSA ended with a larger value of γ than Q-Learning this time. This is a direct contrast to environment 1, where the value of α was higher instead. The reason γ appears higher in environment 2 is because it has no effect on the result of the reward. This is important because there are two rewards now, so the effect of reward should have less of an im-

pact on the route that the agent learns. The reason Q-Learning shows a higher result for α is because it learns its Q-values based on the best action for a given state, so it is more robust to poor decisions. This means a higher value for α will not have as much of an effect if the agent chooses a sub-optimal action.

As you can see, the stochasticity introduced into the rewards made the average reward received by both agents much noisier. This is simply because the value of the reward changes at the start of each episode. Despite this, both agents were able to find the better reward each time. This is evident by the graph showing the average steps for each episode, which is at around 11 steps with no noise at all. Given that the minimum number of steps required to reach the second reward is 13, it is clear that this is not the path that is learned.

Figure 6: The average number of steps of SARSA and Q-Learning in environment 2

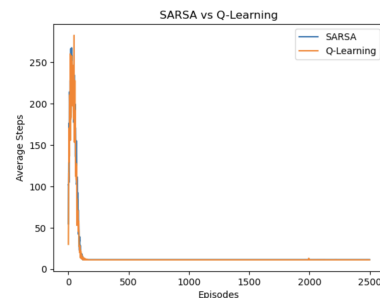


Figure 7: The average reward of SARSA and Q-Learning in environment 2

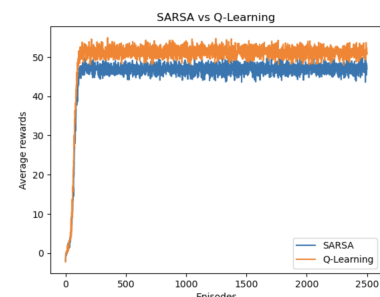


Table 5: A table showing the parameter optimisation for SARSA in environment 3

	Test 1	Test 2	Test 3	Test 4	Test 5	Test 6	Test 7	Test 8
Gamma	0.99	0.75	0.8	0.88	0.9	0.92	0.92	0.95
Alpha	0.3	0.1	0.08	0.07	0.06	0.05	0.04	0.04
Tau	1.5	1.3	1.3	1.2	1.2	1.2	1.2	1.2
Best Path Length	17	13	13	11	14	11	17	11
Final Path Length	40	15	21	19	20	9	16	2

Table 6: A table showing the parameter optimisation for Q-Learning in environment 3

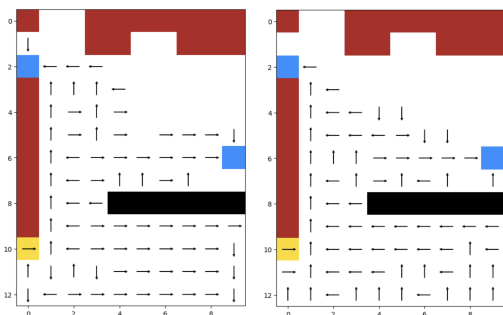
	Test 1	Test 2	Test 3	Test 4	Test 5	Test 6	Test 7	Test 8
Gamma	0.99	0.99	0.7	0.72	0.72	0.72	0.78	0.82
Alpha	0.3	0.2	0.15	0.15	0.15	0.12	0.1	0.08
Tau	1.5	.15	1.3	1.3	1.2	1.2	1.2	1.2
Best Path Length	14	13	11	12	13	11	14	11
Final Path Length	4	12	15	6	17	4	21	6

3.3 Challenge of Stochasticity

The introduction of stochastic rewards presents an interesting challenge for a reinforcement learning agent. If the agent reaches the optimal reward, but is unfortunate enough to recognise it as a poor reward, there is a chance that it may instead learn that the sub-optimal reward is instead better. Given that the better reward has a larger standard deviation than the worse one, the chance of this happening is increased.

The preferred directions learned by SARSA and Q-Learning respectively are shown below.

Figure 8: The preferred directions of SARSA and Q-Learning after training



3.4 Modifications

Tabular eligibility traces could be implemented in order to introduce a form of short term memory beyond just the q-values. An eligibility trace is updated just before the q-values are up-

dated. It is then multiplied by the δ term of the update q-values equation. This means that the more recent positive rewards will have a greater impact on future actions. If tuned correctly, this would amplify the q-values along the best paths, refining the final route more consistently.

An Artificial Neural Network (ANN) is a viable option a q-table replacement, however they are more useful for problems with continuous actions as opposed to the discrete, deterministic actions presented in this environment.

4. Environment 3 - Stochastic Rewards and Transitions

4.1 Overview

The final environment includes the stochastic rewards from environment 2, with the addition of stochastic transitions. In order to simulate the robot slipping as it moves, there is a chance that the agent will remain in the same position.

4.2 Parameter Optimisation

I used the same methodology as described in section 2.2. Tables 5 and 6 show how the parameters changed as I underwent the tuning process.

4.3 Results

Figure 9: *The average number of steps of SARSA and Q-Learning in environment 3*

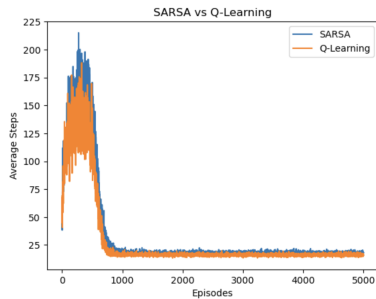
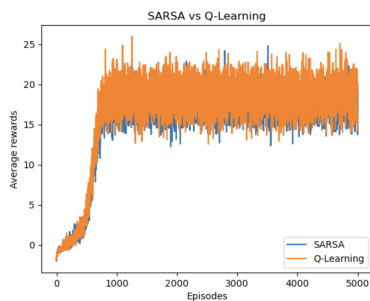


Figure 10: *The average rewards of SARSA and Q-Learning in environment 3*



Compared to the earlier environments, the learning curves for environment 3 appear much noisier. Clearly, the addition of random transitions made learning the optimal path much harder.

4.4 Challenge of additional stochasticity

The noisy learning curve outlined in section 4.3 is expected, as the q-value of the selected state-action pair is still updated in this case. Since the current q-value is taken away as part of the update rule, the resulting q-value will be depreciated in this instance. This adds a further layer of unpredictability, which results in the noisy learning curve.

4.5 Modifications

The additional stochasticity in this environment means that an artificial neural network (ANN) would be a beneficial modification to the learning agent. An ANN can be used to replace the q-value table, so that instead of looking up specific values, the next action can be selected by a pass through the neural network. The updates to the q-values would instead come in the form of updates to the weights and biases of the individual neurons. This would help to combat some of the stochasticity and overall make the learning much smoother and more consistent.

A. Code Snippets

Figure A1: Variable initialisation: Both SARSA and Q-Learning are identical here

```
# Initialise your variables
self.states = env.get_state_size()
self.actions = env.get_action_size()
self.Q = np.zeros((self.states, self.actions))

self._alpha = alpha #Learning rate
self._gamma = gamma #Discount
self._epsilon = epsilon #Explore rate
self.tau = tau #Temperature

total_steps = []
policy_lst = []
total_rewards = []
```

Figure A2: *epsilon_greedy_policy()*

```
def epsilon_greedy_policy(self, state):
    #Determine optimal action(s)
    max_value = np.max(self.Q[state])
    optimal_actions = []
    for action in range(self.actions):
        if self.Q[state, action] == max_value:
            optimal_actions.append(action)

    #Determine whether the agent follows the optimal action or chooses to explore
    if np.random.rand() < self._epsilon:
        return np.random.choice(4) #Explore
    else:
        return np.random.choice(optimal_actions) #Exploit
```


Figure A3: *softmax_policy()*

```
def softmax_policy(self, state):
    q = self.Q[state] #Get q-values of current state
    p = np.zeros(4, dtype=np.float64)

    #Find numerator of softmax function
    for a in range(self.actions):
        n = np.e**(q[a]/self.tau)

    #Find denominator of softmax function
    d = 0
    for b in range(self.actions):
        d += np.e**(q[b]/self.tau)

    #Compute softmax probability
    actionProb = n/d
    p[a] = actionProb

    #Select action according to probabilities
    prob = np.random.uniform(0.0, np.sum(p))
    actionProb = 0
    for action, a in enumerate(p):
        actionProb += a
        if prob < actionProb:
            return action
    return np.random.choice(4)
```

Figure A4: SARSA: *update_Q()*

```
def update_Q(self, state, action, next_state, next_action, reward):
    #Q(s, a) ← Q(s, a) + η * [r + γ * Q(s', a') - Q(s, a)] == SARSA
    self.Q[state, action] += self._alpha * (reward + self._gamma*self.Q[next_state, next_action] - self.Q[state, action])
```

Figure A5: Q-Learning: *update_Q()*

```
def update_Q(self, state, action, next_state, reward):
    #Q(s, a) ← Q(s, a) + η * [r + γ * max_a' Q(s', a') - Q(s, a)] == Q-Learning
    self.Q[state, action] += self._alpha * (reward + self._gamma*np.max(self.Q[next_state]) - self.Q[state, action])
```

Figure A6: SARSA: episode loop

```
for e in range(n_episodes):
    #Initialise episode
    _, state, reward, done = env.reset()
    current_total_steps = 0
    current_policy = [state]
    rewards = []
    rewards.append(reward)

    #Select first action
    if policy == 'EPSILON':
        action = self.epsilon_greedy_policy(state)
    elif policy == 'SOFTMAX':
        action = self.softmax_policy(state)

    while not done:
        _, next_state, reward, done = env.step(action) #Take selected action

        #Select action
        if policy == 'EPSILON':
            next_action = self.epsilon_greedy_policy(next_state)
        elif policy == 'SOFTMAX':
            next_action = self.softmax_policy(next_state)

        self.update_Q(state, action, next_state, next_action, reward) #Update Q-values
        state = next_state #Update current state
        action = next_action #Update current action

        #Record step data
        rewards.append(reward)
        current_policy.append(state)
        current_total_steps += 1

    #Record episode data
    total_rewards.append(np.average(rewards))
    total_steps.append(current_total_steps)
    policy_lst.append(current_policy)
```

Figure A7: Q-Learning: episode loop

```
for e in range(n_episodes):

    #Initialise run
    _, state, reward, done = env.reset()
    current_total_steps = 0
    current_policy = [state]
    rewards = []
    rewards.append(reward)
    while not done:

        #Determine next action
        if policy == 'EPSILON':
            action = self.epsilon_greedy_policy(state)
        elif policy == 'SOFTMAX':
            action = self.softmax_policy(state)

        _, next_state, reward, done = env.step(action) #Take action
        self.update_Q(state, action, next_state, reward) #Update Q-values
        state = next_state #Update current state

        #Record step data
        rewards.append(reward)
        current_policy.append(state)
        current_total_steps += 1

    #Record episode data
    total_rewards.append(np.average(rewards))
    total_steps.append(current_total_steps)
    policy_lst.append(current_policy)
```

Figure A8: `plot_directions()`

```
arrow_len = 0.5
def plot_directions(env, agent):
    fig, ax = plt.subplots(figsize=(8,8))
    grid = get_grid(env)
    ax.imshow(grid)
    q_values = agent.Q #Get q-values
    for state_x in range(env.get_gridshape()[0]):
        for state_y in range(env.get_gridshape()[1]):
            state_coords = [state_x, state_y]
            try:
                state_no = env._get_state_from_loc(state_coords)+1
                state = q_values[state_no-1]
                state_coords = [state_y, state_x]
                max_val = np.max(state)

                #Get number of instances of the best value
                instances = 0
                for i in state:
                    if i == max_val:
                        instances += 1

                #Only show states with a specific preferred direction
                if instances != 1:
                    pass

                #Draw arrows
                elif np.argmax(state) == 0:
                    plt.arrow(state_coords[0], state_coords[1]+(arrow_len/2), 0, -arrow_len, head_length=arrow_len/6, head_width=arrow_len/6)
                elif np.argmax(state) == 1:
                    plt.arrow(state_coords[0]-(arrow_len/2), state_coords[1], arrow_len, 0, head_length=arrow_len/6, head_width=arrow_len/6)
                elif np.argmax(state) == 2:
                    plt.arrow(state_coords[0], state_coords[1]-(arrow_len/2), 0, arrow_len, head_length=arrow_len/6, head_width=arrow_len/6)
                elif np.argmax(state) == 3:
                    plt.arrow(state_coords[0]+(arrow_len/2), state_coords[1], -arrow_len, 0, head_length=arrow_len/6, head_width=arrow_len/6)
            except:
                pass
```